

```
1 package com.tp.jpaa;  
2  
3 import com.tp.jpaa.model.entities.Categoria;  
4 import com.tp.jpaa.model.entities.Producto;  
5 import com.tp.jpaa.repository.CategoriaRepository;  
6 import com.tp.jpaa.repository.ProductoRepository;  
7 import com.tp.jpaa.util.DataLoader;  
8  
9 import java.util.Optional;  
10 import java.util.Scanner;  
11 import java.util.logging.Level;  
12  
13 import static com.tp.jpaa.util.Reports.*;  
14 import static com.tp.jpaa.util.Input.*;  
15 import static com.tp.jpaa.util.Validator.  
    validarNombreCategoria;  
16 import static com.tp.jpaa.util.Validator.  
    validarNombreProducto;  
17  
18  
19 public class Main {  
20     public static void main(String[] args) {  
21  
22  
23         //quita los logs de hibernate dejando solo  
        los errores.  
24         java.util.logging.Logger.getLogger("org.  
        hibernate").setLevel(Level.SEVERE);  
25  
26         Scanner sc = new Scanner(System.in);  
27  
28         CategoriaRepository categoriaRepo = new  
        CategoriaRepository();  
29         ProductoRepository productoRepo = new  
        ProductoRepository();  
30  
31         //Clase solo para pruebas - carga categorias  
        y productos si la base se encuentra vacia.  
32         DataLoader.load(categoriaRepo, productoRepo);  
33  
34         int opcion;
```

```

35
36         do {
37             System.out.println("\n");
38             System.out.println("==== MENÚ PRINCIPAL
===== \n.....");
39             System.out.println("1. Gestión de
Categorías");
40             System.out.println("2. Gestión de
Productos");
41             System.out.println("3. Reportes");
42             System.out.println("0. Salir");
43             System.out.println(
"..... \n");
44
45             opcion = intSeguro(sc, "Ingrese un numero
: ");
46
47             switch (opcion) {
48
49                 case 1 -> menuCategorias(sc,
categoriaRepo);
50
51                 case 2 -> menuProductos(sc,
productoRepo, categoriaRepo);
52
53                 case 3 -> {
54
55                     System.out.println("\n---
REPORTES ---");
56                     System.out.println("Elija una
opcion: ");
57
58                     System.out.println("1. Listar
Productos por categoria");
59                     System.out.println("0. Volver al
menu principal");
60
61                     int opcionReporte = intSeguro(sc
, "Ingrese un numero: \n");
62
63                     switch (opcionReporte) {

```

```

64         case 1 -> {
65
66             if (!
mostrarCategoriasActivas(categoriaRepo)) {
67                 System.out.println("
Operación cancelada.");
68                 break;
69             }
70
71             Long idReporte =
LongSeguro(sc, "\nSeleccione ID de categoría: ");
72
73             listarProductoPorCategoriaHelper(productoRepo,
idReporte);
74
75
76             }
77             case 0 -> System.out.println
("Volviendo al menú principal...");
78
79             }
80
81             }
82
83             case 0 -> System.out.println("
Saliendo del sistema...");
84
85             default -> System.out.println("
Opción inválida.");
86             }
87
88             } while (opcion != 0);
89
90             sc.close();
91         }
92
93         private static void menuCategorias(Scanner sc,
CategoriaRepository categoriaRepo) {
94
95             int opcion;

```

```

96
97         do {
98             System.out.println("\n");
99             System.out.println("\n==== MENÚ DE
CATEGORÍAS ====");
100             System.out.println("1. Alta de categoría
");
101             System.out.println("2. Baja lógica de
categoría");
102             System.out.println("3. Modificación de
categoría");
103             System.out.println("4. Listado de
categorías activas");
104             System.out.println("0. Volver al Menu
Principal");
105             System.out.println(
".....\n");
106
107             opcion = intSeguro(sc, "Ingrese un
numero: ");
108
109
110             switch (opcion) {
111
112                 case 1 -> { //Alta de categoría
113                     System.out.println("\n--- ALTA
DE CATEGORÍA ---");
114                     System.out.print("Nombre: ");
115                     String nombre = sc.nextLine();
116
117                     if (nombre.isBlank()) {
118                         System.out.println("No se
puede crear una categoria sin nombre. Operación
cancelada.");
119                         break;
120                     }
121
122                     if (validarNombreCategoria(
nombre, categoriaRepo, sc)) {
123                         break; // si devuelve true
no continua

```

```

124         }
125
126         System.out.print("Descripción: "
127     );
128         String descripcion = sc.nextLine
129     ();
130
131         Categoria nueva = Categoria.
132     builder()
133         .nombre(nombre)
134         .descripcion(descripcion
135     )
136         .build();
137
138         nueva = categoriaRepo.guardar(
139     nueva);
140
141         System.out.println("Categoría
142     creada con ID: " + nueva.getId());
143     }
144
145     case 2 -> { //Baja logica de
146     Categoria
147         System.out.println("\n--- BAJA
148     LÓGICA DE CATEGORÍA ---");
149
150         if (!mostrarCategoriasActivas(
151     categoriaRepo)) {
152             System.out.println("
153     Operación cancelada.");
154             break;
155         }
156
157         System.out.print("Elija una
158     categoria ");
159
160         //ingreso seguro de long
161         Long id = LongSeguro(sc, "
162     Seleccione ID:");
163
164     }
165     }
166 }

```

```

153
154         Optional<Categoria> optionalCat
    = categoriaRepo.buscarPorId(id);
155
156         if (optionalCat.isEmpty() ||
    optionalCat.get().isEliminado()) {
157             System.out.println("No
    existe una categoría activa con ese ID.");
158         } else {
159             Categoria cat = optionalCat.
    get();
160             boolean eliminado =
    categoriaRepo.eliminarLogico(id);
161
162             if (eliminado) {
163                 System.out.println("
    Categoría dada de baja: " + cat.getNombre());
164             } else {
165                 System.out.println("No
    se pudo eliminar la categoría.");
166             }
167         }
168     }
169
170     case 3 -> { //modificar categoria
171         System.out.println("\n---
    MODIFICACIÓN DE CATEGORÍA ---");
172
173         if (!mostrarCategoriasActivas(
    categoriaRepo)) {
174             System.out.println("No hay
    categorías activas. Operación cancelada.");
175             break;
176         }
177
178         //revisar
179
180         Long id = LongSeguro(sc, "
    Ingrese ID para modificar la categoría: ");
181
182         Optional<Categoria> optionalCat

```

```
182 = categoriaRepo.buscarPorId(id);
183
184         if (optionalCat.isEmpty() ||
optionalCat.get().isEliminado()) {
185             System.out.println("No
existe una categoría activa con ese ID.");
186             break;
187         }
188
189         Categoria cat = optionalCat.get
190 ();
191
192         System.out.println("Valores
actuales:");
193         System.out.println("Nombre: " +
cat.getNombre());
194         System.out.println("Descripción
: " + cat.getDescripcion());
195
196         System.out.print("Nuevo nombre (
ENTER para mantener: ");
197         String nuevoNombre = sc.nextLine
198 ();
199         if (validarNombreCategoria(
nuevoNombre, categoriaRepo, sc)) {
200             break; // salir del case sin
pedir descripción
201         }
202         if (!nuevoNombre.isBlank()) cat.
setNombre(nuevoNombre);
203
204         System.out.print("Nueva
descripción (ENTER para mantener: ");
205         String nuevaDesc = sc.nextLine
206 ();
207         if (!nuevaDesc.isBlank()) cat.
setDescription(nuevaDesc);
208         categoriaRepo.guardar(cat);
```

```
209
210             System.out.println("Categoría
modificada correctamente.");
211         }
212
213         case 4 -> {
214             // Listar categorias
215             System.out.println("\n---
LISTADO DE CATEGORÍAS ACTIVAS ---");
216
217
218             if (!mostrarCategoriasActivas(
categoriaRepo)) {
219                 System.out.println("No hay
categorías activas. Operación cancelada.");
220
221             }
222
223         }
224
225
226         case 0 -> System.out.println("
Volviendo al menú principal...");
227         default -> System.out.println("
Opción inválida.");
228     }
229
230
231     } while (opcion != 0);
232
233 }
234
235     private static void menuProductos(Scanner sc,
ProductoRepository productoRepo, CategoriaRepository
categoriaRepo) {
236
237         int opcion;
238
239         do {
240             System.out.println("\n");
241             System.out.println("\n==== MENÚ DE
```



```

241 PRODUCTOS =====");
242         System.out.println("1. Alta de producto"
    );
243         System.out.println("2. Baja lógica de
    producto");
244         System.out.println("3. Modificación de
    producto");
245         System.out.println("4. Listado de
    productos activos");
246         System.out.println("0. Volver al Menu
    Principal");
247         System.out.println(
    ".....\n");
248         opcion = intSeguro(sc, "Ingrese un
    numero: ");
249
250         switch (opcion) {
251
252             case 1 -> { //Alta de producto
253                 System.out.println("\n--- ALTA
    DE PRODUCTO ---");
254
255
256                 if (!mostrarCategoriasActivas(
    categoriaRepo)) {
257                     System.out.println("
    Operación cancelada.");
258                     break;
259                 }
260
261                 System.out.print("Seleccione ID
    de categoría: ");
262                 //verificar long
263                 Long idCat = LongSeguro(sc, "
    Seleccione ID de categoría: ");
264
265                 Optional<Categoria> optionalCat
    = categoriaRepo.buscarPorId(idCat);
266
267
268                 if (optionalCat.isEmpty() ||

```

```

268 optionalCat.get().isEliminado()) {
269     System.out.println("
    Categoría inválida.");
270     break;
271 }
272
273     Categoria categoria =
    optionalCat.get();
274
275     System.out.print("Nombre: ");
276     String nombre = sc.nextLine();
277     if (nombre.isBlank()) {
278         System.out.println("El
    nombre no puede estar vacío.");
279         break;
280     }
281     if (validarNombreProducto(nombre
    , productoRepo, sc)) {
282         break; // salir del case sin
    pedir descripción
283     }
284
285     System.out.print("Descripción: "
    );
286     String descripcion = sc.nextLine
    ();
287
288     System.out.print("Precio: ");
289     double precio = DoubleSeguro(sc
    , "Ingrese el precio: ");
290
291     if (precio <= 0) {
292         System.out.println("El
    precio debe ser mayor a 0.");
293         break;
294     }
295
296     System.out.print("Stock: ");
297     int stock = intSeguro(sc, "
    Ingrese el stock: ");
298

```

```

299         if (stock < 0) {
300             System.out.println("El stock
           no puede ser negativo.");
301             break;
302         }
303
304         Producto nuevo = Producto.
           builder()
305             .nombre(nombre)
306             .descripcion(descripcion
           )
307             .precio(precio)
308             .stock(stock)
309             .disponible(true)
310             .categoria(categoria)
311             .build();
312
313         nuevo = productoRepo.guardar(
           nuevo);
314
315         System.out.println("Producto
           creado con ID: " + nuevo.getId());
316         System.out.println("Categoría
           asignada: " + categoria.getNombre());
317     }
318
319     case 2 -> { //Baja lógica de
           producto
320         System.out.println("\n--- BAJA
           LÓGICA DE PRODUCTO ---");
321
322
323         if (!mostrarProductosActivos(
           productoRepo)) {
324             System.out.println("
           Operación cancelada.");
325             break;
326         }
327
328         Long id = LongSeguro(sc, "
           Seleccione ID de producto: ");

```

```

329
330         Optional<Producto> optionalProd
    = productoRepo.buscarPorId(id);
331
332         if (optionalProd.isEmpty() ||
    optionalProd.get().isEliminado()) {
333             System.out.println("No
    existe un producto activo con ese ID.");
334             break;
335         }
336
337         Producto prod = optionalProd.get
    ();
338
339         boolean eliminado = productoRepo
    .eliminarLogico(id);
340
341         if (eliminado) {
342             System.out.println("Producto
    dado de baja: " + prod.getNombre());
343         } else {
344             System.out.println("No se
    pudo eliminar el producto.");
345         }
346     }
347
348     case 3 -> { //Modificación de
    producto
349         System.out.println("\n---
    MODIFICACIÓN DE PRODUCTO ---");
350
351
352         if (!mostrarProductosActivos(
    productoRepo)) {
353             System.out.println("
    Operación cancelada.");
354             break;
355         }
356
357         Long id = LongSeguro(sc, "
    Seleccione ID de producto: ");

```

```
358
359         Optional<Producto> optionalProd
    = productoRepo.buscarPorId(id);
360
361         if (optionalProd.isEmpty() ||
    optionalProd.get().isEliminado()) {
362             System.out.println("No
    existe un producto activo con ese ID.");
363             break;
364         }
365
366         Producto prod = optionalProd.get
    ();
367
368         System.out.println("Valores
    actuales:");
369         System.out.println("Nombre: " +
    prod.getNombre());
370         System.out.println("Precio: " +
    prod.getPrecio());
371         System.out.println("Descripción
    : " + prod.getDescripcion());
372         System.out.println("Stock: " +
    prod.getStock());
373
374         System.out.print("Nuevo nombre (
    ENTER para mantener): ");
375         String nuevoNombre = sc.nextLine
    ();
376         if (!nuevoNombre.isBlank()) {
377             if (validarNombreProducto(
    nuevoNombre, productoRepo, sc)) {
378                 break;
379             }
380             prod.setNombre(nuevoNombre);
381         }
382
383
384         System.out.print("Nuevo precio (
    ENTER para mantener): ");
385         String inputPrecio = sc.nextLine
```

```

385 ();
386
387         if (!inputPrecio.isBlank()) {
388             double precio;
389
390             try {
391                 precio = Double.
parseDouble(inputPrecio); //intenta parsearlo
392             } catch (
NumberFormatException e) {
393                 //si falla llama la
funcion doubleseguro
394                 precio = DoubleSeguro(sc
, "Ingrese el nuevo precio, recuerde utilizar solo
numeros : ");
395             }
396
397             if (precio <= 0) {
398                 System.out.println("El
precio debe ser mayor a 0.");
399                 break;
400             }
401
402             prod.setPrecio(precio);
403         }
404
405         System.out.print("Nueva
descripción (ENTER para mantener): ");
406         String nuevaDescripcion = sc.
nextLine();
407         if (!nuevaDescripcion.isBlank
()) prod.setDescripcion(nuevaDescripcion);
408
409         System.out.print("Nuevo stock (
ENTER para mantener): ");
410         String inputStock = sc.nextLine
();
411
412         if (!inputStock.isBlank()) {
413             int stock;
414

```

```

415                try {
416                    stock = Integer.parseInt
(inputStock); //intenta parsearlo
417                } catch (
NumberFormatException e) {
418                    //si falla llama la
funcion intseguro
419                    stock = intSeguro(sc, "
Ingrese el nuevo stock, recuerde utilizar solo
numeros : ");
420                }
421
422                if (stock < 0) {
423                    System.out.println("El
stock no puede ser negativo.");
424                    break;
425                }
426
427                prod.setStock(stock);
428            }
429
430            productoRepo.guardar(prod);
431
432            System.out.println("Producto
modificado correctamente.");
433        }
434
435        case 4 -> { //Listado de productos
activos
436            System.out.println("\n---
LISTADO DE PRODUCTOS ACTIVOS ---");
437
438            mostrarProductosActivos(
productoRepo);
439
440
441        }
442
443
444        // Volver
445        case 0 -> System.out.println("

```

```
445 Volviendo al menú principal...");
446         default -> System.out.println("
        Opción inválida.");
447     }
448
449     } while (opcion != 0);
450 }
451
452 }
453
454
455
456
457
458
```



```
1 package com.tp.jp.a.util;
2
3 import java.util.Scanner;
4
5 public class Input {
6     public static Double DoubleSeguro(Scanner sc,
7     String mensaje) {
8         while (true) {
9             System.out.print(mensaje);
10            String input = sc.nextLine();
11
12            try {
13                return Double.parseDouble(input);
14            } catch (NumberFormatException e) {
15                System.out.println("Debe ingresar un
16                número válido.");
17            }
18        }
19    }
20
21    public static Long LongSeguro(Scanner sc, String
22    mensaje) {
23        while (true) {
24            System.out.print(mensaje);
25            String input = sc.nextLine();
26
27            try {
28                return Long.parseLong(input);
29            } catch (NumberFormatException e) {
30                System.out.println("Debe ingresar un
31                número válido.");
32            }
33        }
34    }
35
36    public static int intSeguro(Scanner sc, String
37    mensaje) {
38        while (true) {
39            System.out.print(mensaje);
40            String input = sc.nextLine();
41
42            try {
43                return Integer.parseInt(input);
44            } catch (NumberFormatException e) {
45                System.out.println("Debe ingresar un
46                número válido.");
47            }
48        }
49    }
50}
```

```
37         try {
38             return Integer.parseInt(input);
39         } catch (NumberFormatException e) {
40             System.out.println("Debe ingresar un
        número válido.");
41         }
42     }
43 }
44 }
```

```
1 package com.tp.jp.util;  
2  
3 import jakarta.persistence.EntityManager;  
4 import jakarta.persistence.EntityManagerFactory;  
5 import jakarta.persistence.Persistence;  
6  
7 public class JPAUtil {  
8  
9     private static final String PERSISTENCE_UNIT = "  
miUnidad";  
10  
11     private static EntityManagerFactory emf;  
12  
13     static {  
14         try {  
15             emf = Persistence.  
createEntityManagerFactory(PERSISTENCE_UNIT);  
16         } catch (Exception e) {  
17             System.err.println("Error inicializando  
EntityManagerFactory: " + e.getMessage());  
18             throw new ExceptionInInitializerError(e);  
19         }  
20     }  
21  
22     public static EntityManager getEntityManager() {  
23         return emf.createEntityManager();  
24     }  
25  
26     //como usarlo -> EntityManager em = JPAUtil.  
getEntityManager();  
27  
28     public static EntityManagerFactory  
getEntityManagerFactory() {  
29         return emf;  
30     }  
31  
32     public static void close() {  
33         if (emf != null && emf.isOpen()) {  
34             emf.close();  
35         }  
36     }
```

37 }

38

```
1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 import java.util.List;
9
10 public class Reports {
11     public static void
12     listarProductoPorCategoriaHelper(ProductoRepository
13     productoRepo, long idReporte){
14
15         List<Producto> productos = productoRepo.
16         buscarPorCategoria(idReporte);
17
18         if (productos.stream().findAny().isEmpty()) {
19             System.out.println("No hay productos
20             activos para la categoría seleccionada.");
21         } else {
22             System.out.println("Productos de la
23             categoria: ");
24             productos.forEach(p ->
25                 System.out.println(
26                     "ID: " + p.getId() +
27                     " | Nombre: " + p
28                     .getNombre() +
29                     " | Precio: " + p
30                     .getPrecio() +
31                     " | Descripcion
32                     : " + p.getDescripcion() +
33                     " | Stock: " + p.
34                     getStock() +
35                     " | Categoría: "
36                     + p.getCategoria().getNombre()
37                     )
38             );
39         }
40     }
41 }
```

```
32     public static boolean mostrarProductosActivos(  
    ProductoRepository productoRepo) {  
33  
34         List <Producto> productos = productoRepo.  
listarActivos();  
35  
36         if (productos.stream().findAny().isEmpty()) {  
37             System.out.println("No hay productos  
activos.");  
38             return false;  
39         } else {  
40             productos.forEach(p ->  
41                 System.out.println(  
42                     "ID: " + p.getId() +  
43                     " | Nombre: " + p  
.getNombre() +  
44                     " | Precio: " + p  
.getPrecio() +  
45                     " | Descripcion  
: " + p.getDescripcion() +  
46                     " | Stock: " + p.  
getStock() +  
47                     " | Categoría: "  
+ p.getCategoria().getNombre()  
48                     )  
49                 );  
50             return true;  
51         }  
52     }  
53  
54     public static boolean mostrarCategoriasActivas(  
    CategoriaRepository categoriaRepo) {  
55  
56         List<Categoria> categorias = categoriaRepo.  
listarActivos();  
57  
58         if (categorias.stream().findAny().isEmpty  
()) {  
59             System.out.println("No hay categorías  
activas.");  
60             return false;
```

```
61         } else {
62             System.out.println("Categorias
disponibles :");
63             categorias.forEach(cat ->
64                 System.out.println(
65                     "ID: " + cat.getId() +
66                     " | Nombre: " +
cat.getNombre() +
67                     " | Descripcion
: " + cat.getDescripcion()
68                 )
69             );
70             return true;}
71     }
72
73 }
```

```
1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 import java.util.Optional;
9 import java.util.Scanner;
10
11 import static com.tp.jpa.util.Input.intSeguro;
12
13 public class Validator {
14
15     public static boolean validarNombreCategoria(
16         String nombre, CategoriaRepository categoriaRepo,
17         Scanner sc) {
18         //Validacion de nombre duplicado
19         boolean existe = categoriaRepo.listarActivos
20             ()
21             .stream()
22             .anyMatch(c -> c.getNombre().
23                 equalsIgnoreCase(nombre));
24
25         Optional<Categoria> categoriaInactiva =
26             categoriaRepo.listarInactivos()
27             .stream()
28             .filter(c -> c.getNombre().
29                 equalsIgnoreCase(nombre))
30             .findFirst();
31
32         //verificacion de categoria repetida
33         if (existe) {
34             System.out.println("Ya existe una
35                 categoría con ese nombre.");
36             return true; // no continua
37         }
38
39         //verificacion de categoria existente pero
40         con baja logica
41         if (categoriaInactiva.isPresent()) {
42             System.out.println("Ya existe una
```



```

33 categoría con ese nombre, pero se encuentra inactiva"
    +
34         "\n ¿Desea activarla nuevamente"
    );
35         System.out.println("1. Si");
36         System.out.println("2. No");
37         int opcionAlta = intSeguro(sc, "
    Seleccione una opción: ");
38
39         switch (opcionAlta) {
40             case 1 -> {
41                 Categoria cat = categoriaInactiva
42                     .get();
43                 categoriaRepo.AltaLogica(cat.
44                     getId());
45                 System.out.println("Categoría
    reactivada correctamente. ID: " + cat.getId());
46                 return true;
47             }
48             case 2 -> {
49                 System.out.println("Operación
    cancelada.");
50                 return true; //no continua
51             }
52         }
53     }
54     return false; //si pasa las evaluaciones
    continua con el alta
55 }
56
57
58 public static boolean validarNombreProducto(
    String nombre, ProductoRepository productoRepo,
    Scanner sc) {
59     boolean existe = productoRepo.listarActivos()
60         .stream()
61         .anyMatch(c -> c.getNombre().
    equalsIgnoreCase(nombre));
62

```

```

63         Optional<Producto> prodInactivo =
            productoRepo.listarInactivos()
64             .stream()
65             .filter(c -> c.getNombre().
                equalsIgnoreCase(nombre))
66             .findFirst();
67
68         //verificacion de producto repetido
69         if (existe) {
70             System.out.println("Ya existe un
producto con ese nombre.");
71             return true; // no continua
72         }
73
74         //verificacion de producto existente pero
            con baja logica
75         if (prodInactivo.isPresent()) {
76             System.out.println("Ya existe un
producto con ese nombre, pero se encuentra inactivo"
77             +
                "\n ¿Desea activarlo nuevamente"
78             );//
79             System.out.println("1. Si");
80             System.out.println("2. No");
81             int opcionAltaProd = intSeguro(sc, "
Seleccione una opción: ");
82
83             switch (opcionAltaProd) {
84                 case 1 -> {
85                     Producto prod = prodInactivo.get
86                     ();
87                     productoRepo.AltaLogica(prod.
88                     getId());
89                     System.out.println("Producto
reactivado correctamente. ID: " + prod.getId());
90                     return true; // no continua
91                 }
92                 case 2 -> {
93                     System.out.println("Operación
cancelada.");
94                     return true; // no continua

```

```
92         }
93     }
94
95
96     }
97     return false; //si pasa las evaluaciones
    continua con el alta
98 }
99 }
100
```

```
1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 public class DataLoader {
9
10     public static void load(CategoriaRepository
        categoriaRepo, ProductoRepository productoRepo) {
11
12         //ESTA CLASE ES SOLO PARA PRUEBAS DEL CODIGO
        . CARGA CATEGORIAS Y PRODUCTOS SI LA BASE SE
        ENCUENTRA VACIA.
13         // Si ya existen categorías o productos
        cargados, no carga nada
14         boolean hayCategorias = !categoriaRepo.
        listarActivos().isEmpty() ||
15         !categoriaRepo.listarInactivos().
        isEmpty();
16
17         boolean hayProductos = !productoRepo.
        listarActivos().isEmpty() ||
18         !productoRepo.listarInactivos().
        isEmpty();
19
20         if (hayCategorias || hayProductos) {
21             return;
22         }
23
24         System.out.println("Cargando datos
        iniciales de hardware...");
25
26         // =====
27         // ===== 1. CATEGORÍAS =====
28         // =====
29         Categoria micro = Categoria.builder()
30             .nombre("Microprocesadores")
31             .descripcion("CPUs Intel y AMD")
32             .build();
```

```

33
34         Categoria mother = Categoria.builder()
35             .nombre("Motherboards")
36             .descripcion("Placas madre Intel y
AMD")
37             .build();
38
39         Categoria memorias = Categoria.builder()
40             .nombre("Memorias RAM")
41             .descripcion("DDR4 y DDR5")
42             .build();
43
44         Categoria perifericos = Categoria.builder()
45             .nombre("Periféricos")
46             .descripcion("Teclados, mouse,
auriculares")
47             .build();
48
49         Categoria fuentes = Categoria.builder()
50             .nombre("Fuentes de alimentación")
51             .descripcion("PSU 500W a 1000W")
52             .build();
53
54         micro = categoriaRepo.guardar(micro);
55         mother = categoriaRepo.guardar(mother);
56         memorias = categoriaRepo.guardar(memorias);
57         perifericos = categoriaRepo.guardar(
perifericos);
58         fuentes = categoriaRepo.guardar(fuentes);
59
60         // =====
61         // 2. PRODUCTOS
62         // =====
63
64         // MICROPROCESADORES
65         productoRepo.guardar(Producto.builder().
nombre("Intel i3").descripcion("4 núcleos, 8 hilos").
precio(95000.0).stock(15).disponible(true).categoria(
micro).build());
66         productoRepo.guardar(Producto.builder().
nombre("Intel i5").descripcion("6 núcleos, 12 hilos"

```

```
66 ).precio(150000.0).stock(10).disponible(true).
    categoria(micro).build());
67     productoRepo.guardar(Producto.builder().
    nombre("Intel i7").descripcion("12 núcleos, 20 hilos
    ").precio(280000.0).stock(8).disponible(true).
    categoria(micro).build());
68     productoRepo.guardar(Producto.builder().
    nombre("Ryzen 5").descripcion("6 núcleos, 12 hilos"
    ).precio(130000.0).stock(12).disponible(true).
    categoria(micro).build());
69     productoRepo.guardar(Producto.builder().
    nombre("Ryzen 7").descripcion("8 núcleos, 16 hilos"
    ).precio(210000.0).stock(7).disponible(true).
    categoria(micro).build());
70
71     // MOTHERBOARDS
72     productoRepo.guardar(Producto.builder().
    nombre("ASUS Prime").descripcion("Intel 12va gen").
    precio(120000.0).stock(10).disponible(true).
    categoria(mother).build());
73     productoRepo.guardar(Producto.builder().
    nombre("Gigabyte B550M").descripcion("AM4 Ryzen").
    precio(110000.0).stock(14).disponible(true).
    categoria(mother).build());
74     productoRepo.guardar(Producto.builder().
    nombre("MSI B450").descripcion("AM4 Ryzen").precio(
    105000.0).stock(9).disponible(true).categoria(mother
    ).build());
75     productoRepo.guardar(Producto.builder().
    nombre("ASRock Z690").descripcion("Intel 12va gen").
    precio(220000.0).stock(5).disponible(true).categoria
    (mother).build());
76     productoRepo.guardar(Producto.builder().
    nombre("ASUS ROG").descripcion("AM4 Gaming").precio(
    180000.0).stock(6).disponible(true).categoria(mother
    ).build());
77
78     // MEMORIAS RAM
79     productoRepo.guardar(Producto.builder().
    nombre("Corsair 8GB").descripcion("3200MHz").precio(
    25000.0).stock(20).disponible(true).categoria(
```

```
79 memorias).build());
80     productoRepo.guardar(Producto.builder().
    nombre("Corsair 16GB").descripcion("3200MHz").precio
    (45000.0).stock(18).disponible(true).categoria(
    memorias).build());
81     productoRepo.guardar(Producto.builder().
    nombre("HyperX 16GB").descripcion("3600MHz").precio(
    50000.0).stock(15).disponible(true).categoria(
    memorias).build());
82     productoRepo.guardar(Producto.builder().
    nombre("Kingston 8GB").descripcion("5200MHz").precio
    (60000.0).stock(10).disponible(true).categoria(
    memorias).build());
83     productoRepo.guardar(Producto.builder().
    nombre("Kingston 16GB").descripcion("5600MHz").
    precio(90000.0).stock(8).disponible(true).categoria(
    memorias).build());
84
85     // PERIFÉRICOS
86     productoRepo.guardar(Producto.builder().
    nombre("Logitech G203").descripcion("Mouse gamer RGB
    ").precio(15000.0).stock(25).disponible(true).
    categoria(perifericos).build());
87     productoRepo.guardar(Producto.builder().
    nombre("Redragon K552").descripcion("Teclado
    mecánico").precio(30000.0).stock(20).disponible(true)
    ).categoria(perifericos).build());
88     productoRepo.guardar(Producto.builder().
    nombre("HyperX Cloud").descripcion("Auriculares
    gamer").precio(35000.0).stock(12).disponible(true).
    categoria(perifericos).build());
89     productoRepo.guardar(Producto.builder().
    nombre("Logitech C270").descripcion("Webcam HD").
    precio(20000.0).stock(10).disponible(true).categoria
    (perifericos).build());
90     productoRepo.guardar(Producto.builder().
    nombre("Redragon M607").descripcion("Mouse gamer
    7200 DPI").precio(12000.0).stock(18).disponible(true)
    ).categoria(perifericos).build());
91
92     // FUENTES
```

```
93         productoRepo.guardar(Producto.builder().
           nombre("Corsair CV550").descripcion("550W 80+ Bronze
           ").precio(35000.0).stock(10).disponible(true).
           categoria(fuentes).build());
94         productoRepo.guardar(Producto.builder().
           nombre("EVGA 600 BR").descripcion("600W 80+ Bronze"
           ).precio(40000.0).stock(8).disponible(true).
           categoria(fuentes).build());
95         productoRepo.guardar(Producto.builder().
           nombre("Gigabyte P650B").descripcion("650W 80+
           Bronze").precio(45000.0).stock(7).disponible(true).
           categoria(fuentes).build());
96         productoRepo.guardar(Producto.builder().
           nombre("Thermaltake 700W").descripcion("700W 80+
           White").precio(50000.0).stock(6).disponible(true).
           categoria(fuentes).build());
97         productoRepo.guardar(Producto.builder().
           nombre("Corsair RM750").descripcion("750W 80+ Gold"
           ).precio(80000.0).stock(5).disponible(true).
           categoria(fuentes).build());
98
99         System.out.println("Datos iniciales de
           hardware cargados correctamente.");
100
101     }
102 }
103
```



```
1 package ar.edu.tup.programacion3.dtos;
2
3 public record UsuarioDTO(
4     String nombre,
5     String apellido,
6     String mail,
7     String celular) {
8     public UsuarioDTO {
9         if (nombre == null) {
10             nombre = "Sin nombre";
11         } else if (apellido == null) {
12             apellido = "Sin apellido";
13         } else if (mail == null) {
14             mail = "Sin mail";
15         } else if (celular == null) {
16             celular = "Sin celular";
17         }
18     }
19 }
20 }
21
22
```

```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum Rol {  
4     ADMIN, USUARIO  
5 }  
6
```

```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum Estado {  
4     PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO  
5 }  
6
```

```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum FormaPago {  
4     TARJETA, TRANSFERENCIA, EFECTIVO  
5 }  
6
```

```
1 package com.tp.jpa.model.entities;
2 import jakarta.persistence.GeneratedValue;
3 import jakarta.persistence.GenerationType;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.MappedSuperclass;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8 import java.time.LocalDateTime;
9
10 @MappedSuperclass
11 @Getter
12 @Setter
13 @ToString
14 @SuperBuilder
15 @AllArgsConstructor
16 @NoArgsConstructor
17 @EqualsAndHashCode(of = {"id"})
18 public abstract class Base {
19
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.
        IDENTITY)
23     private Long id;
24     @Builder.Default
25     protected boolean eliminado = false;
26     @Builder.Default
27     protected LocalDateTime createdAt = LocalDateTime
        .now();
28 }
```

```
1 package com.tp.jpa.model.entities;
2
3 import ar.edu.tup.programacion3.enums.Estado;
4 import ar.edu.tup.programacion3.enums.FormaPago;
5 import jakarta.persistence.*;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8
9 import java.time.LocalDate;
10 import java.util.HashSet;
11 import java.util.Set;
12 @Entity
13 @Table(name = "pedidos")
14 @Getter
15 @Setter
16 @ToString
17 @EqualsAndHashCode(callSuper = true)
18 @AllArgsConstructor
19 @NoArgsConstructor
20 @SuperBuilder
21 public class Pedido extends Base implements
    Calculable {
22     private LocalDate fecha;
23     @Enumerated(EnumType.STRING)
24     private Estado estado;
25     private Double total;
26     @Enumerated(EnumType.STRING)
27     private FormaPago formapago;
28     @ManyToOne
29     @JoinColumn(name = "usuario_id")
30     private Usuario usuario;
31
32     //coleccion de detalle pedidos
33     @Builder.Default
34     @OneToMany(mappedBy = "pedido", cascade =
        CascadeType.ALL, orphanRemoval = true)
35     private Set<DetallePedido> detallePedidos = new
        HashSet<>();
36
37
38     //Agregar detalle
```

```
39     public void addDetallePedido(int cantidad,
    Producto producto) {
40
41         boolean exists = detallePedidos.stream()
42             .anyMatch(detallePedido ->
    detallePedido.getProducto().equals(producto));
43         if (exists) return;
44
45         DetallePedido detallePedido = DetallePedido.
    builder()
46             .cantidad(cantidad)
47             .producto(producto)
48             .subtotal(producto.getPrecio()*
    cantidad)
49             .pedido(this)
50             .build();
51         detallePedidos.add(detallePedido);
52
53     }
54
55     //Buscar detalle por producto
56     public DetallePedido findDetallePedidoByProducto(
    Producto producto) {
57         return detallePedidos.stream()
58             .filter(detallePedido ->
    detallePedido.getProducto().equals(producto))
59             .findFirst()
60             .orElse(null);
61     }
62
63
64     //Eliminar detalle por producto
65     public void deleteDetalleByProducto(Producto
    producto) {
66         detallePedidos.removeIf(d -> d.getProducto().
    equals(producto));
67     }
68
69
70     @Override
71     public void calcularTotal() {
```

```
72         double totalFinal = detallePedidos.stream()
73             .mapToDouble(DetallePedido::
                getSubtotal)
74             .sum();
75         this.total = totalFinal;
76         System.out.println("Total del pedido: " +
            totalFinal);
77     }
78
79
80
81 }
```



```
1 package com.tp.jpamodel.entities;
2 import ar.edu.tup.programacion3.enums.Rol;
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.*;
8
9 @Entity
10 @Table(name = "usuarios")
11 @Getter
12 @Setter
13
14 @ToString(callSuper = true, exclude = "contrasenia")
15 @EqualsAndHashCode(callSuper = false, of = {"mail"})
16
17 @SuperBuilder
18 @NoArgsConstructor
19 @AllArgsConstructor
20 public class Usuario extends Base {
21     private String nombre;
22     private String apellido;
23     @Column(unique = true)
24     private String mail;
25     @Column(unique = true)
26     private String celular;
27     private String contrasenia;
28     @Enumerated(EnumType.STRING)
29     private Rol rol;
30
31     //sets de pedidos
32
33
34     @Builder.Default
35     @OneToMany(mappedBy = "usuario")
36     private Set<Pedido> pedidos = new HashSet<>();
37     public void addPedido(Pedido pedido) {
38         pedidos.add(pedido);
39     }
40
41
```

42

43 }

```
1 package com.tp.jpamodel.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 @Entity
8 @Table(name = "productos")
9 @Getter
10 @Setter
11 @ToString
12 @EqualsAndHashCode(callSuper = false , of = {"nombre"
13     , "precio"})
13 @SuperBuilder
14 @NoArgsConstructor
15 @AllArgsConstructor
16 public class Producto extends Base {
17     @Column(unique = true)
18     private String nombre;
19     private Double precio;
20     private String descripcion;
21     private int stock;
22     private String imagen;
23     private boolean disponible;
24     @ManyToOne
25     @JoinColumn(name = "categoria_id")
26     private Categoria categoria;
27
28
29 }
```

```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.HashSet;
8 import java.util.Set;
9
10 @Entity
11 @Table(name = "categorias")
12 @SuperBuilder
13 @Getter
14 @Setter
15 @ToString(callSuper = true, exclude = "productos")
16 @EqualsAndHashCode(callSuper = false, of = {"nombre"
    })
17 @AllArgsConstructor
18 @NoArgsConstructor
19 public class Categoria extends Base {
20     @Column(unique = true)
21     private String nombre;
22     private String descripcion;
23     @OneToMany(mappedBy = "categoria", cascade =
        CascadeType.ALL, orphanRemoval = true)
24     @Builder.Default
25     private Set<Producto> productos = new HashSet
        <>();
26
27     //helper para mantener la bidireccionalidad
28     public void addProducto(Producto producto) {
29         productos.add(producto);
30         producto.setCategoria(this);
31     }
32 }
33
34
35
36
```

```
1 package com.tp.jpa.model.entities;
2
3 public interface Calculable{
4     void calcularTotal();
5 }
6
7
```

```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.JoinColumn;
5 import jakarta.persistence.ManyToOne;
6 import jakarta.persistence.Table;
7 import lombok.*;
8 import lombok.experimental.SuperBuilder;
9
10 @Entity
11 @Table(name = "detalle_pedidos")
12 @Getter
13 @Setter
14 @EqualsAndHashCode(callSuper = false, of = {"producto
    "})
15 @NoArgsConstructor
16 @AllArgsConstructor
17 @SuperBuilder
18 public class DetallePedido extends Base {
19
20     private int cantidad;
21     private Double subtotal;
22     @ManyToOne
23     @JoinColumn(name = "producto_id")
24     private Producto producto;
25     @ManyToOne
26     @JoinColumn(name = "pedido_id", nullable = false)
27     private Pedido pedido;
28
29     public DetallePedido(int cantidad, Double
    subtotal) {
30         this.cantidad = cantidad;
31         this.subtotal = subtotal;
32
33     }
34 }
```

```
1 package com.tp.jpa.repository;
2 import com.tp.jpa.model.entities.Base;
3 import com.tp.jpa.util.JPAUtil;
4 import jakarta.persistence.EntityManagerFactory;
5
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public abstract class BaseRepository<T extends Base
    > {
11
12     protected final Class<T> entityClass;
13     protected final EntityManagerFactory emf;
14
15     public BaseRepository(Class<T> entityClass) {
16         this.entityClass = entityClass;
17         this.emf = JPAUtil.getEntityManagerFactory();
18     }
19
20     // ===== 1. Guardar (T entity) =====
21     public T guardar(T entity) {
22         var em = emf.createEntityManager();
23         var tx = em.getTransaction();
24
25         try {
26             tx.begin();
27             T merged = em.merge(entity);
28             tx.commit();
29             return merged;
30         } catch (Exception e) {
31             if (tx.isActive()) {
32                 tx.rollback();
33             }
34             throw new RuntimeException("Error al
guardar la entidad en BaseRepository", e);
35         } finally {
36             em.close();
37         }
38     }
39
```

```
40 // ===== 2. buscarPorId(Long id) =====
41 public Optional<T> buscarPorId(Long id) {
42     var em = emf.createEntityManager();
43
44     try {
45         T entity = em.find(entityClass, id);
46         return Optional.ofNullable(entity); //
        retorna empty si es null
47
48     } finally {
49         em.close();
50     }
51 }
52
53 // ===== 3. listarActivos =====
54 public List<T> listarActivos() {
55     var em = emf.createEntityManager();
56
57     try {
58         String jpql = "SELECT e FROM " +
        entityClass.getSimpleName() + " e WHERE e.eliminado
        = false";
59         return em.createQuery(jpql, entityClass).
        getResultList();
60
61     } finally {
62         em.close();
63     }
64 }
65
66 // ===== 3. listarInactivos =====
67 public List<T> listarInactivos() {
68     var em = emf.createEntityManager();
69
70     try {
71         String jpql = "SELECT e FROM " +
        entityClass.getSimpleName() + " e WHERE e.eliminado
        = true";
72         return em.createQuery(jpql, entityClass).
        getResultList();
73 }
```



```

74         } finally {
75             em.close();
76         }
77     }
78
79
80     // ===== 4. eliminarLogico =====
81
82     public boolean eliminarLogico(Long id) {
83         var em = emf.createEntityManager();
84         var tx = em.getTransaction();
85
86         try {
87             T entity = em.find(entityClass, id);
88
89             if (entity == null) {
90                 return false; //mo existe -> retorna
false
91             }
92
93             tx.begin();
94             entity.setEliminado(true);
95             em.merge(entity);
96             tx.commit();
97
98             return true;
99
100         } catch (Exception e) {
101             if (tx.isActive()) {
102                 tx.rollback();
103             }
104             throw new RuntimeException("Error al
eliminar lógicamente la entidad", e);
105
106         } finally {
107             em.close();
108         }
109     }
110
111     // Alta logixa
112

```

```
113     public boolean AltaLogica(Long id) {
114         var em = emf.createEntityManager();
115         var tx = em.getTransaction();
116
117         try {
118             T entity = em.find(entityClass, id);
119
120             if (entity == null) {
121                 return false; //no existe -> retorna
122                 falso
123             }
124
125             tx.begin();
126             entity.setEliminado(false);
127             em.merge(entity);
128             tx.commit();
129
130             return true;
131         } catch (Exception e) {
132             if (tx.isActive()) {
133                 tx.rollback();
134             }
135             throw new RuntimeException("Error al dar
136             de alta la entidad la entidad", e);
137         } finally {
138             em.close();
139         }
140     }
141
142
143 }
144
```

```
1 package com.tp.jpa.repository;
2
3 import com.tp.jpa.model.entities.Producto;
4 import jakarta.persistence.EntityManager;
5
6 import java.util.List;
7
8 public class ProductoRepository extends
  BaseRepository<Producto> {
9
10     public ProductoRepository() {
11         super(Producto.class);
12     }
13
14
15     // Consulta JPQL que obtiene todos los productos
    activos de una categoría
16     // filtrando por p.categoria.id = :categoriaId y
    p.eliminado = false.
17     public List<Producto> buscarPorCategoria(Long
    categoriaId) {
18         var em = emf.createEntityManager();
19
20         try {
21             String jpql = "SELECT p FROM Producto p "
22 +
23             "WHERE p.categoria.id = :
    categoriaId " +
24             "AND p.eliminado = false";
25
26             return em.createQuery(jpql, Producto.
    class)
27                 .setParameter("categoriaId",
    categoriaId)
28                 .getResultList();
29         } finally {
30             em.close();
31         }
32     }
33
```

```
34     public List<Producto> buscarPorNombre(String
      nombre) {
35         EntityManager em = emf.createEntityManager();
36         try {
37             return em.createQuery(
38                 "SELECT p FROM Producto p
      WHERE p.nombre = LOWER(:nombre)",
39                 Producto.class
40             ).setParameter("nombre", nombre)
41             .getResultList();
42         } finally {
43             em.close();
44         }
45     }
46
47 }
48
```

```
1 package com.tp.jp.a.repository;
2
3 import com.tp.jp.a.model.entities.Base;
4 import com.tp.jp.a.model.entities.Categoria;
5
6
7
8 public class CategoriaRepository extends
  BaseRepository<Categoria> {
9
10     public CategoriaRepository() {
11         super(Categoria.class);
12     }
13 }
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/
  persistence"
3         xmlns:xsi="http://www.w3.org/2001/
  XMLSchema-instance"
4         xsi:schemaLocation="https://jakarta.ee/
  xml/ns/persistence
5 https://jakarta.ee/xml/ns/persistence/persistence_3_0
  .xsd" version="3.0">
6     <persistence-unit name="miUnidad"
7         transaction-type="
  RESOURCE_LOCAL">
8         <provider>org.hibernate.jpa.
  HibernatePersistenceProvider</provider>
9         <!-- CLASES -->
10        <class>com.tp.jpa.model.entities.Usuario</
  class>
11        <class>com.tp.jpa.model.entities.Pedido</
  class>
12        <class>com.tp.jpa.model.entities.
  DetallePedido</class>
13        <class>com.tp.jpa.model.entities.Producto</
  class>
14        <class>com.tp.jpa.model.entities.Categoria</
  class>
15
16        <properties>
17            <!-- JDBC H2 en archivo -->
18            <property name="jakarta.persistence.jdbc.
  driver"
19                value="org.h2.Driver"/>
20            <property name="jakarta.persistence.jdbc.
  url"
21                value="jdbc:h2:file:./data/
  jpa_db"/>
22            <property name="jakarta.persistence.jdbc.
  user" value="sa"/>
23            <property name="jakarta.persistence.jdbc.
  password" value=""/>
24            <!-- Dialecto -->
25            <property name="hibernate.dialect"

```

```
26         value="org.hibernate.dialect.  
H2Dialect"/>  
27         <!-- Hibernate -->  
28         <property name="hibernate.hbm2ddl.auto"  
value="update"/>  
29         <!-- Se cambia a false para no mostrar  
los SQL de las busquedas -->  
30         <property name="hibernate.show_sql" value  
="false"/>  
31         <property name="hibernate.format_sql"  
value="false"/>  
32         <property name="hibernate.  
use_sql_comments" value="false"/>  
33     </properties>  
34 </persistence-unit>  
35 </persistence>
```